

EVALUATION · ENGINEERING REFERENCE

Evaluating AI systems from LLMs to agent memory

A layered framework for measuring quality, from raw model output up through retrieval, RAG, multi-step agents, and the memory that lets them persist. Definitions, formulas and reference architectures for engineers.



Application › Retrieval › RAG › Agent › Agent Memory

What evaluation is, and its four lenses

Evaluation is the **systematic measurement of a system's behaviour against a defined expectation**, using metrics that are observable, comparable and reproducible. For AI systems the expectation is never one number: it spans correctness, cost, safety and behaviour over time. The discipline is choosing the right *unit of analysis* for each layer, then measuring it with metrics that isolate that layer's failure modes.

The same word names four different problems. Each narrows the unit of analysis and the question being asked.

■ LLM EVALUATION

Unit · one model call

Intrinsic generation quality: accuracy, helpfulness, safety, instruction following, format adherence, refusal calibration.

Benchmarks, reference scoring, LLM-as-judge.

Is this output correct, safe and on-spec for the prompt?

■ RAG EVALUATION

Unit · retrieve-then-generate

Whether the answer is supported by retrieved evidence, and whether retrieval surfaced the right evidence at all.

Split into retrieval quality and generation faithfulness.

Is the answer grounded in the right context, and did we fetch it?

■ AGENT EVALUATION

Unit · multi-step trajectory

Task completion, tool selection and argument grounding, trajectory efficiency and reasoning quality across steps.

Component, trajectory and outcome scoring.

Did it reach the goal on a sound path?

■ AGENT MEMORY EVAL

Unit · state across sessions

Cross-session recall, knowledge updates, temporal reasoning, abstention and per-user isolation under token budgets.

Write / store / retrieve / update probes.

Does it remember the right things, and forget the rest?

Five layers of evaluation in an agentic application

Each layer wraps the one beneath it. A defect at a lower layer surfaces as a failure at every layer above, so evaluation drills downward: confirm the foundation before blaming the behaviour.

01	Application serving & system	Scope The request/response envelope around any model call.	Fails when Slow first token, p99 latency blowouts, runaway cost.
02	Retrieval search quality	Scope Did the retriever return the right items, ranked well, for a query.	Fails when Relevant evidence missing or buried below the cut-off.
03	RAG grounded generation	Scope Is the generated answer supported by the retrieved context.	Fails when Confident answers unsupported by, or contradicting, the evidence.
04	Agent multi-step behaviour	Scope Planning, tool selection, argument grounding, trajectory to a goal.	Fails when Wrong tool, malformed args, looping, or a bad path to a right-looking answer.
05	Agent Memory state over time	Scope Write, store, retrieve and update facts across turns and sessions.	Fails when Forgets, mis-updates, leaks across users, or fabricates instead of abstaining.

Five layers of evaluation in an agentic application

Each level sits on the one beneath it. A failure low in the stack caps the score of everything above, so you evaluate bottom-up and attribute every regression to the layer that actually caused it.

05



Agent Memory

PERSISTENCE

Does it recall, update and reason over time?

04



Agent

BEHAVIOUR

Did it take the right actions to reach the goal?

03



RAG

GROUNDING

Is the answer grounded in that retrieved context?

02



Retrieval

INFORMATION

Did we fetch the right context into the top-k?

01



Application

FOUNDATION

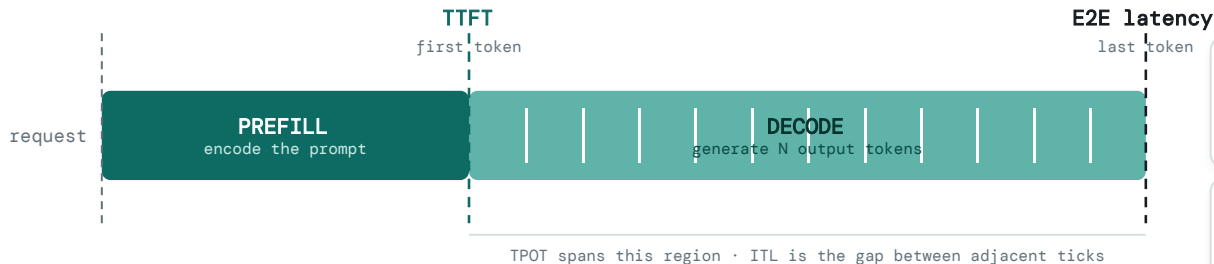
Is it fast, available and affordable under load?

The metric set for each layer, at a glance

01 Application	TTFT TPOT / ITL E2E latency throughput tok/s cost / 1K tok p95 / p99	Speed, scalability and unit economics of serving.
02 Retrieval	Precision@k Recall@k F1 MRR NDCG@k Hit Rate@k MAP	Did the right items come back, ranked near the top.
03 RAG	Faithfulness Groundedness Answer Relevance Context Precision Context Recall Chunk Utilisation Correctness	Is the answer supported, was the evidence sufficient and used.
04 Agent	Task Success Tool-Selection Acc. Arg / Call Acc. Trajectory Match Step Efficiency Cost & Latency / task	Did it finish the job, with the right actions, efficiently.
05 Agent Memory	Cross-session Recall Knowledge Update Temporal Reasoning Abstention Isolation Tokens / query	Right facts at the right time, updated and bounded, no leakage.

Serving latency, throughput and cost

Before any quality metric, the system has to be fast and affordable under load. These are black-box metrics on the request/response envelope, measured at the percentile, never the mean.



End-to-end latency

$$E2E = TTFT + TPOT \cdot (N_{out} - 1)$$

Throughput

$$\text{tok/s} = N_{out} / E2E \cdot RPS$$

■ TTFT

Time to first token. Request to first emitted token, dominated by queueing plus prefill. The headline number for perceived responsiveness.

■ TPOT / ITL

Time per output token, or inter-token latency. The mean gap between successive tokens during decode. Sets streaming speed.

■ THROUGHPUT

Tokens/sec and requests/sec at a target concurrency. Governs how many users a deployment serves; goodput = RPS within SLO.

■ COST

cost = in_tok·price_in + out_tok·price_out. Track \$/request and \$/1K tokens. Caching, batching and routing move this most.

■ RELIABILITY

Availability, error and timeout rate, and SLO attainment at the tail. A fast median with a heavy p99 tail still fails users.

Serving latency, throughput and cost

CORE RELATIONS

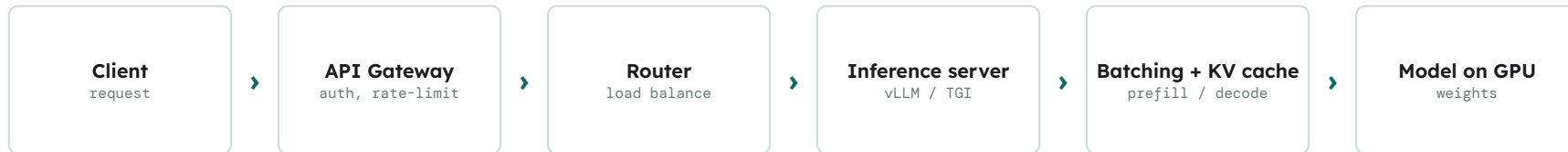
$e2e_latency \approx TTFT + TPOT \cdot N_{out}$

$throughput = output_tokens / wall_clock_s$

$cost/query = N_{in} \cdot price_{in} + N_{out} \cdot price_{out}$

$goodput = requests_meeting_SLO / requests$

THE SERVING PATH · WHERE LATENCY AND COST ACCRUE



Ranking metrics for search quality

Retrieval is an information-retrieval problem. Two questions: did the relevant items come back at all (set quality), and were they ranked near the top (order quality).

RANKED RESULTS · $k = 8$

1	relevant document	●
2	relevant document	●
3	non-relevant	
4	relevant document	●
5	non-relevant	
6	relevant document	●
7	non-relevant	
8	non-relevant	

first relevant at rank 1 → $RR = 1 / 1 = 1.00$

WORKED 5 relevant in corpus, 4 in top-8:
 $P@8 = 4/8 = 0.50$ · $R@8 = 4/5 = 0.80$

Precision@k

$$\text{rel_in_topk} / k$$

Of the k returned, what fraction is relevant. Penalises noise in the cut.

Recall@k

$$\text{rel_in_topk} / \text{rel_total}$$

Of all relevant items, what fraction made the top-k. Penalises misses.

F1

$$2 \cdot P \cdot R / (P + R)$$

Harmonic mean of precision and recall when both matter equally.

MRR

$$\text{mean}(1 / \text{rank_first_rel})$$

Mean reciprocal rank of the first relevant hit. Rewards top placement.

NDCG@k

$$\text{DCG@k} / \text{IDCG@k}$$

Graded, position-discounted gain vs the ideal order. $\text{DCG} = \sum_i \text{rel}_i / \log_2(i+1)$.

Hit Rate@k & MAP

$$\geq 1 \text{ rel in top-k} \cdot \text{mean AP}$$

Hit Rate: any relevant in the cut. MAP averages precision at each relevant rank.

Information retrieval: relevance over a ranked list

Retrieval quality has two faces, are the items we returned relevant (precision) and did we find all the relevant ones (recall), plus a third axis ranking metrics care about: are the relevant items near the top?



QUERY

retriever



RANKED RESULTS (top-k)

rank 1	relevant	●
rank 2	not relevant	
rank 3	relevant	●
rank 4	relevant	●
rank 5	not relevant	
rank 6	not relevant	

top-k cutoff (k=4) ›

RELEVANCE CONFUSION

True Positive
retrieved & relevant

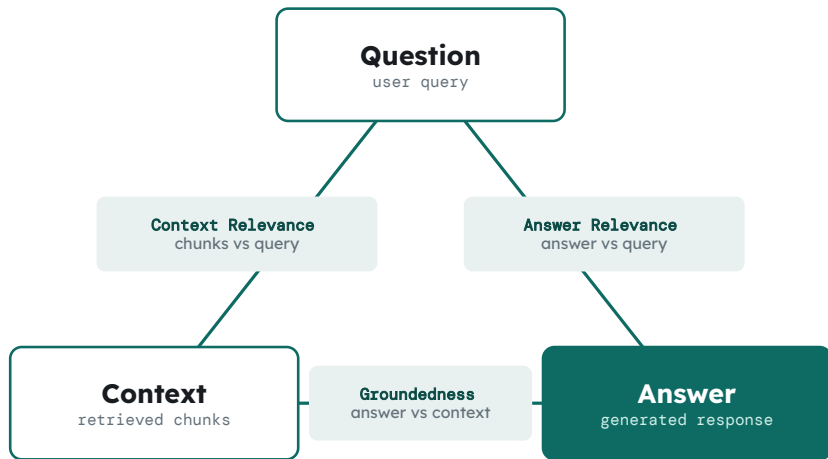
False Positive
retrieved, not relevant

False Negative
missed a relevant item

True Negative
correctly not retrieved

The RAG triad: grounding the answer in evidence

Retrieval-augmented generation is evaluated over a triple of (query, retrieved context, answer). Three edges triangulate quality; six further metrics quantify each edge.



THE SIX MEASURES BEHIND THE EDGES

Faithfulness

$\text{supported_claims} / \text{total_claims}$

Claim-level grounding: every assertion in the answer is entailed by the context.

Context Precision

$\text{signal vs noise in top-k}$

Are the relevant chunks ranked above the irrelevant ones in what was retrieved.

Context Recall

$\text{retrieved_rel} / \text{ground_truth_rel}$

Did retrieval bring back all the evidence the reference answer actually needs.

Chunk Utilisation

$\text{used_context} / \text{retrieved_context}$

Fraction of retrieved context the answer draws on. Flags over-retrieval.

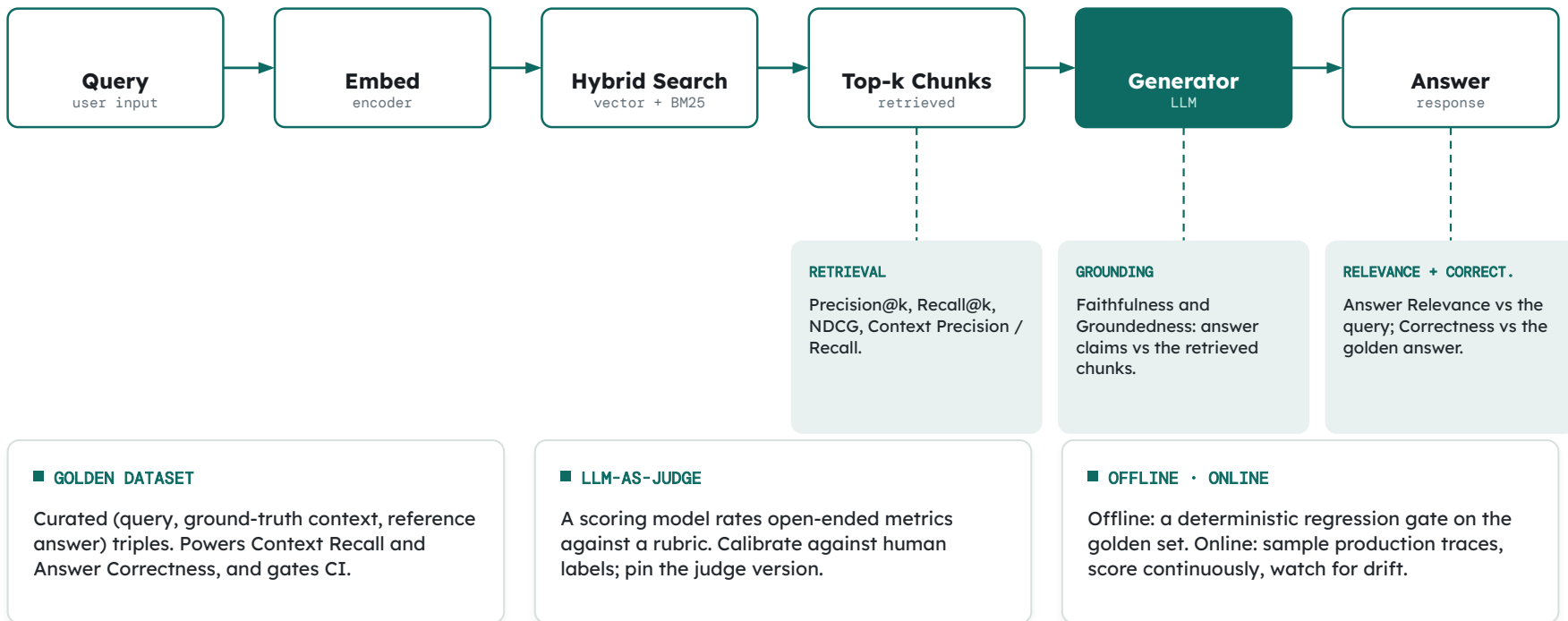
Answer Correctness

$\text{factual} + \text{semantic vs reference}$

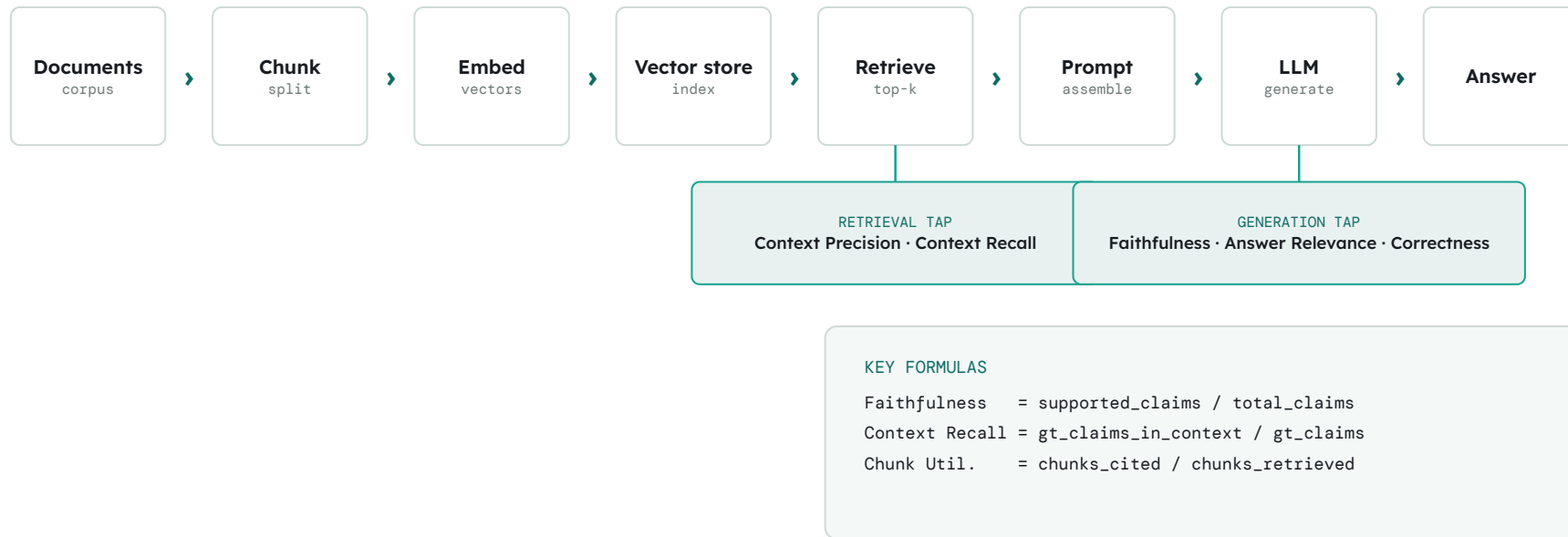
End-to-end correctness against a golden answer, beyond mere grounding.

Evaluation reference architecture for RAG

Instrument the pipeline at every boundary. Each probe isolates one failure mode, so a bad answer can be attributed to retrieval, to grounding, or to the generator.



Evaluation reference architecture for RAG



Agent evaluation: outcome and trajectory

An agent can reach the right answer by a wrong path (lucky), or follow a right path that fails at the end. Outcome metrics catch the former, trajectory metrics catch the latter. Mature evaluation scores both, along two questions: what to judge, and how.

TWO AXES OF AGENT EVALUATION

WHAT

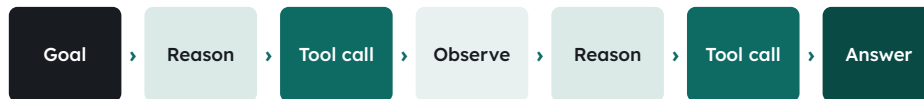
Final response: was the outcome correct?

Trajectory: was the path of actions correct?

HOW

Reference-based match · programmatic checks ·
LLM-as-judge on steps and result

AN EVALUATED TRAJECTORY



- tool selected correct?
- arguments valid?
- step necessary?
- recovered from error?
- goal achieved?

Outcome metrics judge the last box. Trajectory metrics judge the whole chain.

Why memory needs its own evaluation

A long context window is not a memory system. One measures capacity over a fixed input; the other measures continuity across a stream that is written to and read from over time.

LONG CONTEXT

Capacity

One large fixed input, retrieved and reasoned over in a single pass.
Nothing is written; nothing persists after the call.

Fails by attending to the wrong span.

MEMORY SYSTEM

Continuity

A stream over time. State at turn 50 depends on turns 1 to 49: facts are written, stored and retrieved on later turns.

Fails by storing, retrieving or ranking the wrong thing.

Capacity and continuity are different problems. The same score does not measure both.

FIVE PROPERTIES A COMPLETE MEMORY BENCHMARK MUST EXERCISE

01

Cross-session continuity

Surface a fact from an earlier session when a later one needs it. Without this, it is a chat buffer.

02

Selective writes

Decide what is worth keeping. Write everything and it fills; write nothing and there is no memory.

03

Retrieval under budget

Real agents have a token budget. Injecting 50K tokens of memory per prompt is not a memory test.

04

Forgetting & update

Overwrite stale facts, merge duplicates, reconcile contradictions instead of stacking them.

05

Per-user isolation

Keep user A's memory out of user B's answers. A single synthetic user cannot catch this.

Benchmarks: LongMemEval, LongMemEval-V2, BEAM

Three benchmarks define the field in 2026. Each tests multi-session write-and-retrieve, not single-pass attention, and each raises a different bar.

LongMemEval

ICLR 2025

SCOPE

Chat-assistant memory over user histories.

SCALE

500 curated Q. LME-S ~115K tok / ~40 sessions;
LME-M ~500 sessions (~1.5M tok).

CORE ABILITIES

- Information Extraction
- Multi-Session Reasoning
- Temporal Reasoning
- Knowledge Updates
- Abstention

FINDING Long-context LLMs drop ~30% recalling across sustained chats. Framework: index / retrieve / read.

LongMemEval-V2

2026

SCOPE

Long-term memory for web & enterprise agents (WebArena / ServiceNow style).

SCALE

451 curated Q. Up to 500 trajectories & ~115M tokens per haystack; multimodal.

CORE ABILITIES

- Static State Recall
- Dynamic State Tracking
- Workflow Knowledge
- Environment Gotchas
- Premise Awareness

METRIC Scores accuracy and query latency together via LAFS Gain, the accuracy gain on a log-scaled latency frontier (1s-200s).

BEAM

ICLR 2026

SCOPE

“Beyond a Million Tokens”: coherent conversations at extreme scale.

SCALE

100 conversations, 128K-10M tok. 2,000 probing Q across general / coding / math.

CORE ABILITIES

- Fact & entity tracking
- Updates over time
- Contradiction resolution
- Temporal order, multi-hop
- Instruction vs preference

FINDING Ten abilities; tracks BEAM-1M / 10M. A 10M window does not replace memory; LIGHT adds 3.5-12.7%.

■ THE THROUGH-LINE

Evaluate every layer. Attribute every failure.

A green dashboard at the top means nothing if the layer beneath it is red. Measure serving, retrieval, grounding, trajectory and memory as distinct contracts, each with metrics that isolate its own failure modes. When something breaks, read the stack top-down to the layer that actually broke, instead of tuning the symptom.

■ 01 Application

■ 02 Retrieval

■ 03 RAG

■ 04 Agent

■ 05 Agent Memory